

Automatique

Seite Alexander et Nathan Vidonne

February 23, 2026

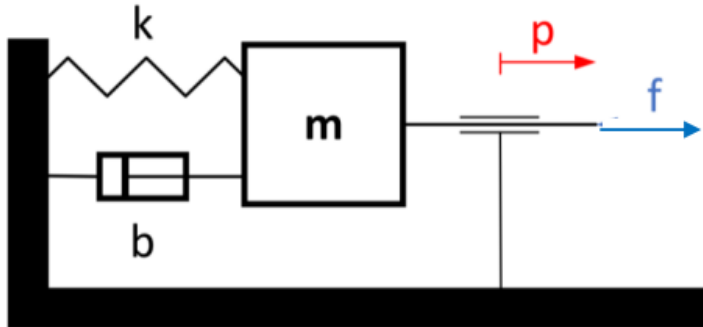
Contents

1	Introduction	2
2	Exercice 1 : Système masse-ressort (matlab)	2
2.1	A : Equation du mouvement du système	2
2.2	B : Fonction de transfert (FT) $G(s)$	2
2.3	C : Code matlab	3
2.4	D : Affichage FT $G(s)$ sous forme d'Evans et de Bode	3
2.5	E : Réponse indicielle à 1N	3
2.6	F : Overshoot, Rising time, Settling time	4
3	Exercice 2 : Système masse-ressort (simulink)	5
4	Exercice 3 : Balance à fléau	7
4.1	A : Schema Simulink	8
4.2	B : Application d'un regulateur proportionnel (P)	9
4.3	C : Changement du comportement induit par K_p	10
4.4	D : Application d'un regulateur integrateur (PI)	11
5	Conclusion	12
6	annexes	13

1 Introduction

2 Exercice 1 : Système masse-ressort (matlab)

k représente la constante de raideur du ressort relié à la masse m qui subit un frottement b lors d'un changement de position p par une force f .



La position p en m

La raideur du ressort $k = 200 \frac{N}{m}$

La viscosité $b = 2 \frac{N}{(m/s)}$

La masse $m = 0.1 \text{ kg}$

La force appliquée au système f en N

2.1 A : Equation du mouvement du système

L'équation différentielle du système est définie à partir de la seconde loi de Newton.

$$\sum F = ma$$

$$\Leftrightarrow m \cdot \ddot{p}(t) = -k \cdot p(t) - b \cdot \dot{p}(t) + f(t)$$

2.2 B : Fonction de transfert (FT) $G(s)$

A partir de l'équation précédente, nous déterminons la fonction de transfert de notre système après avoir réalisé la transformée de Laplace de l'équation du système.

$$\mathcal{L} \rightarrow m \cdot s^2 \cdot P(s) = -k \cdot P(s) - b \cdot s \cdot P(s) + F(s)$$

$$\Leftrightarrow P(s) \cdot (ms^2 + k + b) = F(s)$$

$$\Leftrightarrow G(s) = \frac{1}{ms^2 + k + b}$$

2.3 C : Code matlab

Compléter...

2.4 D : Affichage FT $G(s)$ sous forme d'Evans et de Bode

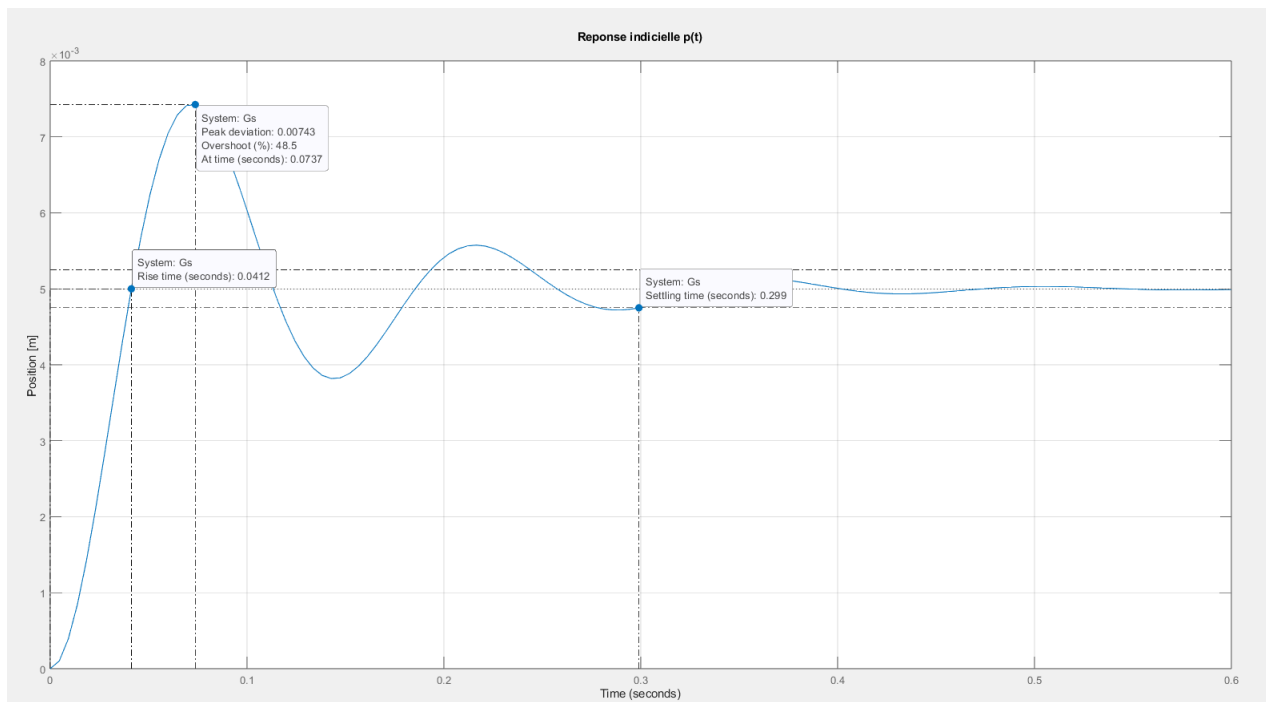
$G(s)$ représenté sous ses formes de Bode et d'Evans. La commande `Gs_zpk_evans` affiche la fonction de transfert sous forme zéros-pôles-gain, utile pour analyser la stabilité et la dynamique du système.

$$Evans \rightarrow \frac{10}{s^2 + 20s + 2000}$$

Les commandes `Gs_zpk_bode` et `Gs_zpk_bode.DisplayFormat;` permettent d'afficher la fonction de transfert sous une forme adaptée à l'analyse fréquentiel

$$Bode \rightarrow \frac{0.005}{(1 + 0.4472(\frac{s}{44.72}) + \frac{s}{44.72})^2}$$

2.5 E : Réponse indicielle à 1N



La commande `step(Gs)` trace la réponse temporelle du système à une entrée en échelon unitaire. Nous pouvons observer la réponse à un échelon de force de 1N.

Grâce aux fonctions graphiques de Simulink, nous observons la réponse indicielle du système avec un Overshoot de 48.5% à 0.0737 secondes. Le rising time est de 0.0412.

2.6 F : Overshoot, Rising time, Settling time

```
Continuous-time zero/pole/gain model.  
Model Properties
```

```
Overshoot =
```

```
48.5150
```

```
RiseTime =
```

```
0.0412
```

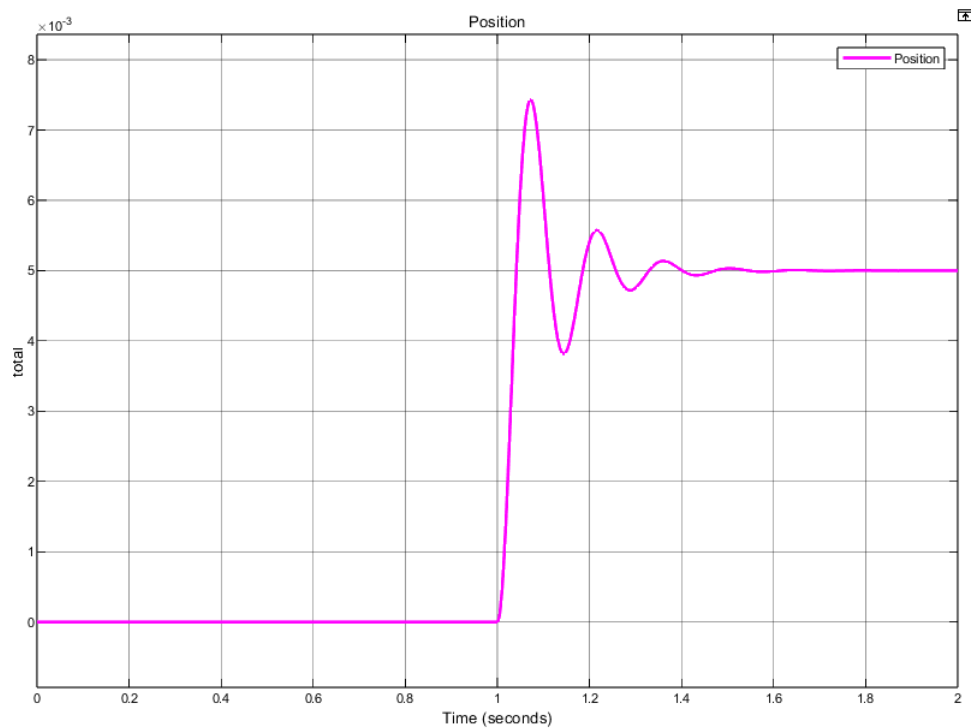
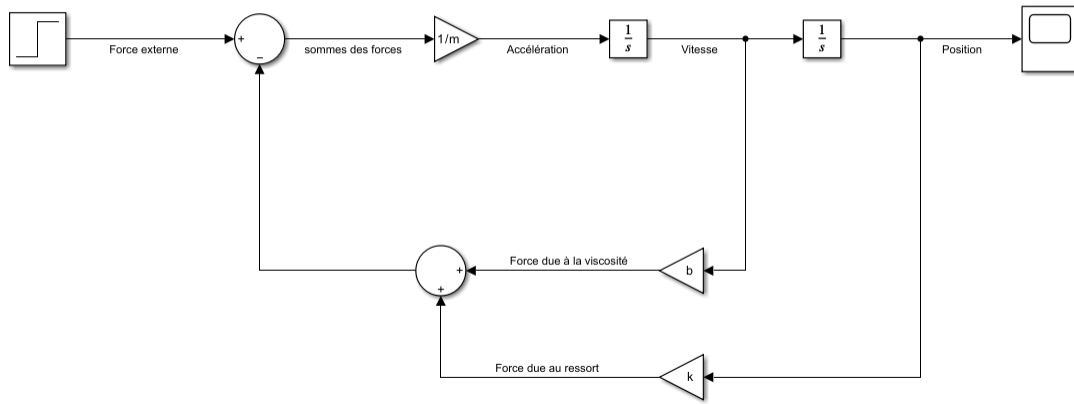
```
SettlingTime =
```

```
0.2990
```

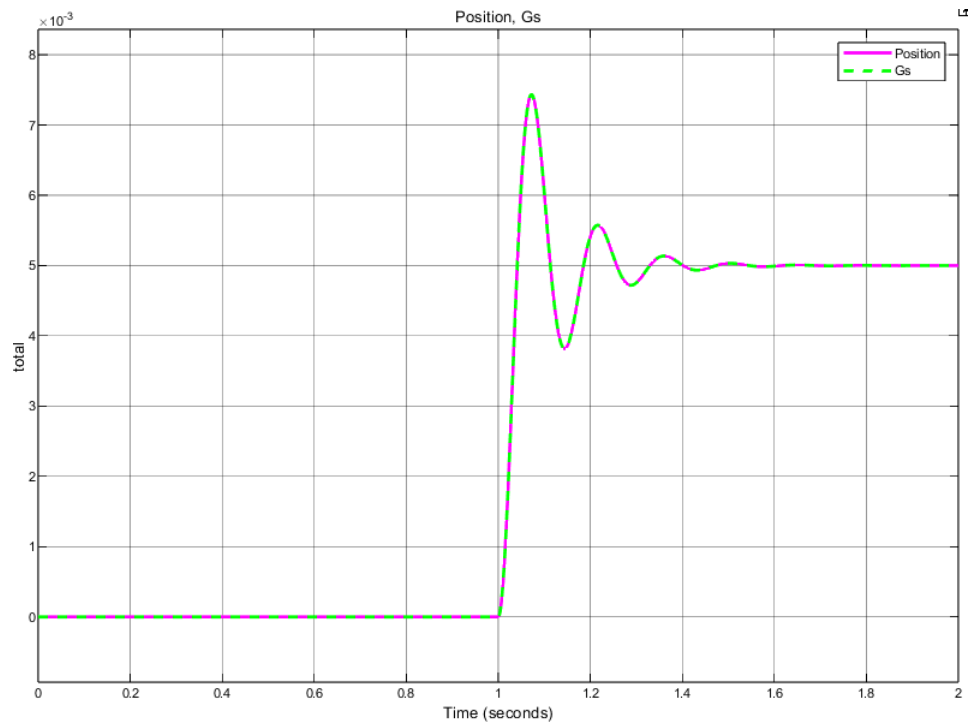
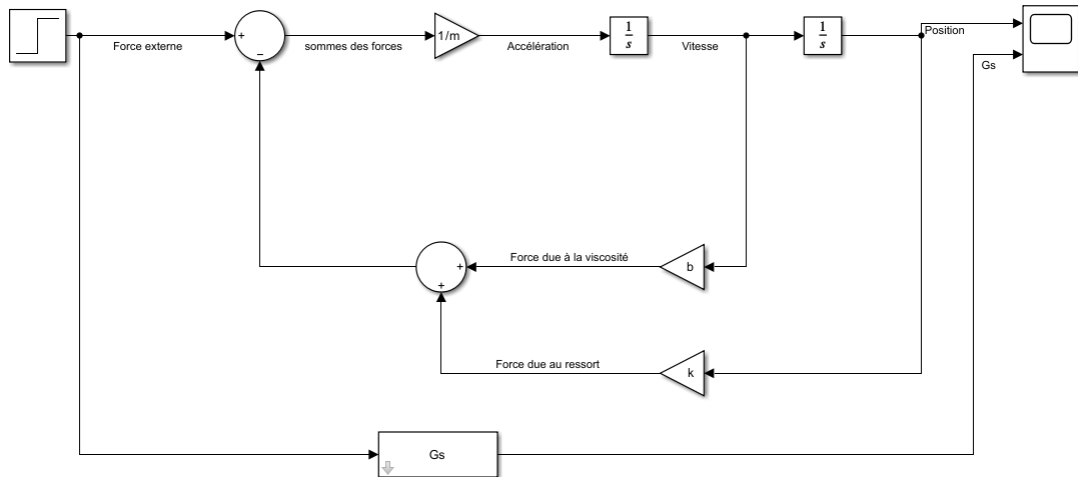
Les données recueillies des paramètres donnés par la fonction `stepinfo` sur labview sont les mêmes que celles sur le graphique (section 2.5 E).

Le Overshoot est un pourcentage par lequel la réponse dépasse la valeur finale. Le RiseTime est le temps nécessaire de 10% à 90% de la valeur finale et le SettlingTime est le temps nécessaire pour que la réponse reste dans une bande de $\pm 5\%$ fde la valeur finale.

3 Exercice 2 : Système masse-ressort (simulink)

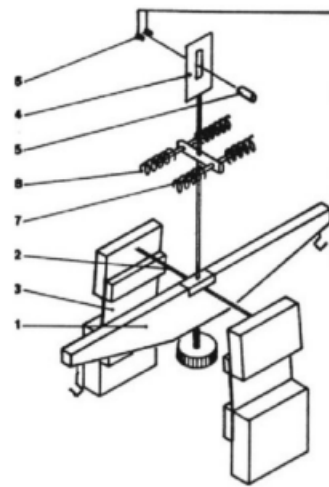


Nous pouvons confirmer que le schéma représente en effet le système étudié, car lors de la simulation nous retrouvons la même courbe que dans l'exercice précédant (section 2.5 E). (soit caractérisé par un amortissement faible, de même amplitude, nombre d'oscillation et gain statique)



Les réponses obtenues sont identiques. Et c'est normal... Le bloc LTI est simplement un bloc logique représentant le système caractérisé par $G(s)$. Un même système aura la même sortie pour une même entrée.

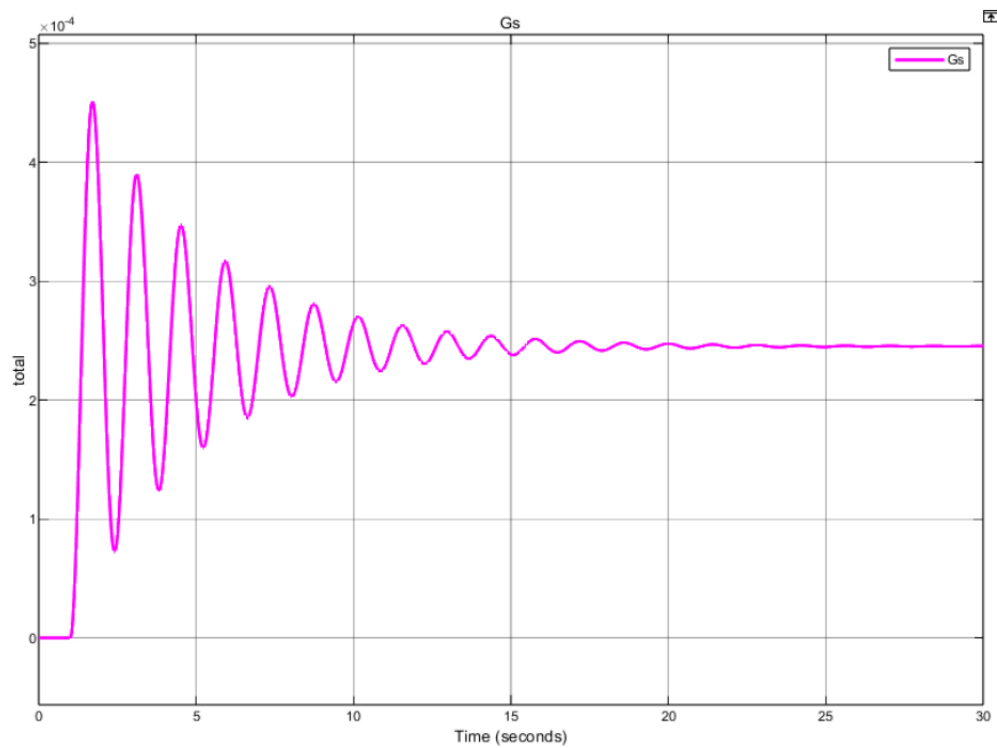
4 Exercice 3 : Balance à fléau



1. Fléau
2. Ruban de torsion
3. Lame de tension du ruban
4. Drapeau
5. Led
6. Eléments photosensibles (phototransistors)
7. Aimants
8. Bobines

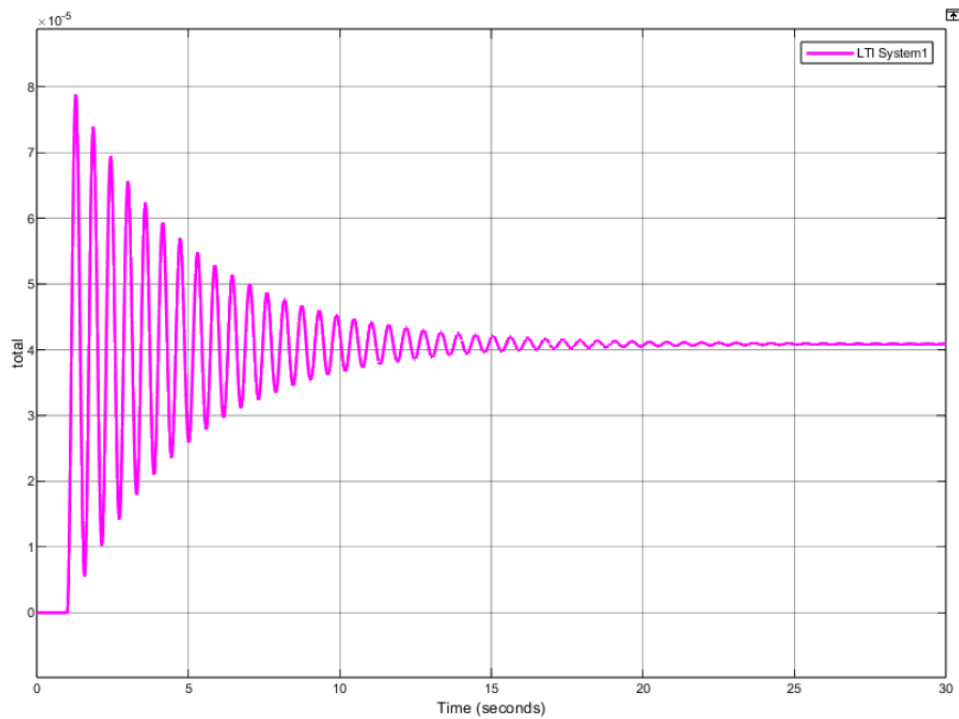
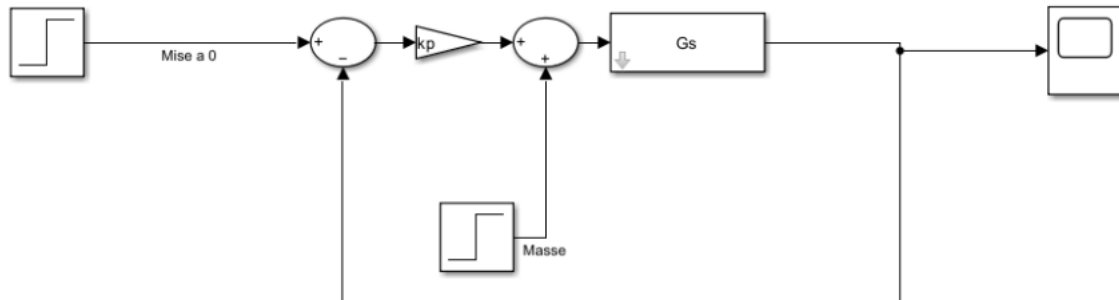
Image is self-explanatory.

4.1 A : Schema Simulink



La courbe du point 3A montre la réponse d'un système sous-amorti à une perturbation brutale. Avant $t=1$ s, le fléau est stable à $= 0$. Quand on ajoute la masse, le système oscille avec une amplitude qui diminue progressivement à cause de l'amortissement, jusqu'à se stabiliser à une nouvelle position d'équilibre. Les oscillations prouvent que le système est sous-amorti (< 1), et l'erreur statique finale montre qu'il ne revient pas exactement à sa position initiale sans correction.

4.2 B : Application d'un regulateur proportionnel (P)

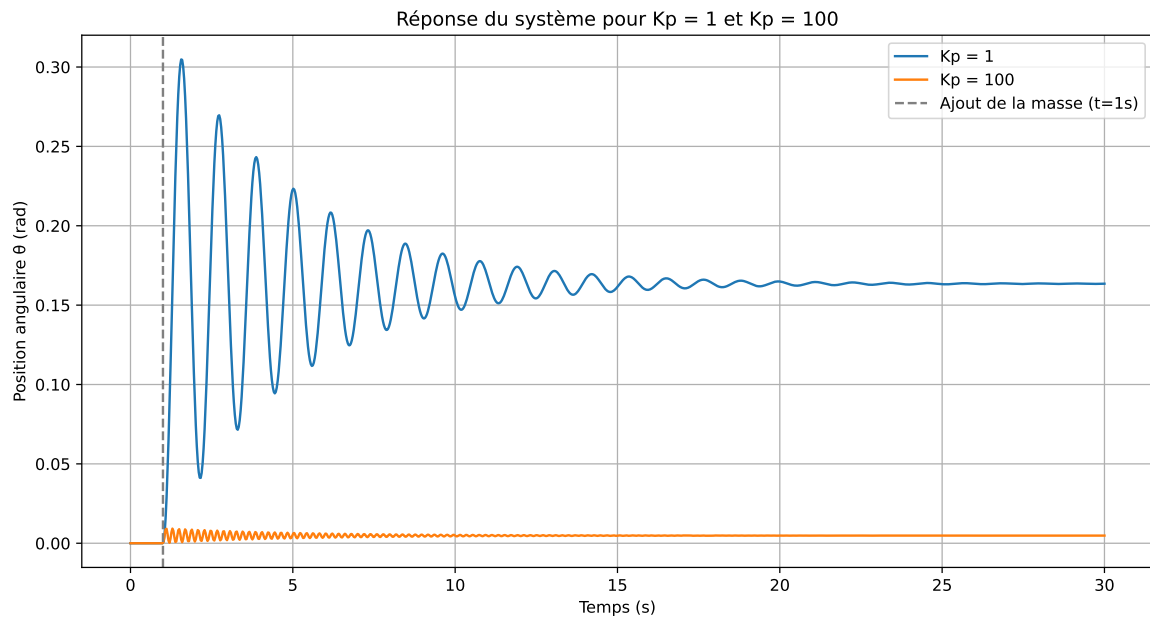


En ajoutant un regulateur proportionnel de gain $K_p = 10$, le système va tenter de rejeter la perturbation (la masse ajoutée). La réponse sera plus rapide et plus stable qu'en boucle ouverte, mais il peut y avoir un erreur statique (la position ne revient pas exactement à 0 rad)

4.3 C : Changement du comportement induit par K_p

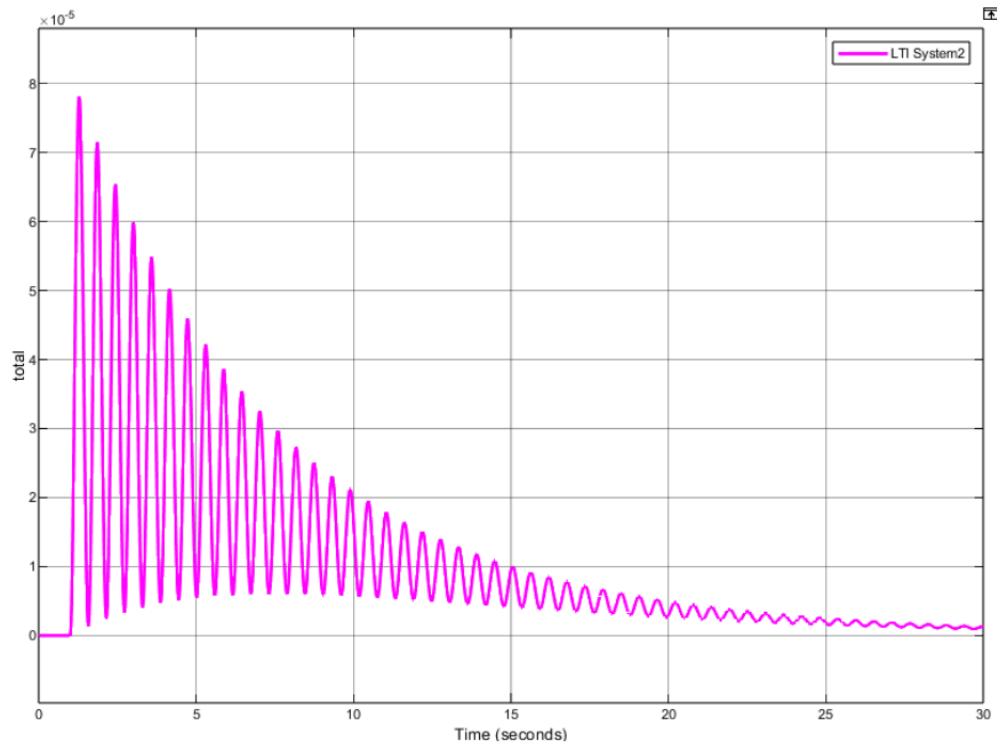
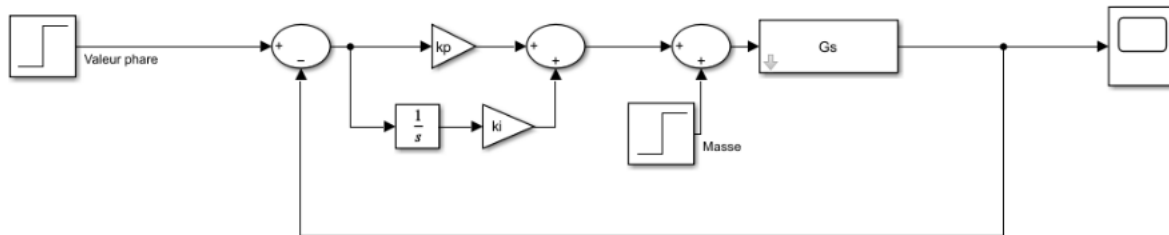
Nous avons oublié de run la simulation pour différents gains du régulateur K_p . Toutefois, nous pouvons affirmer que la variation de K_p impacte directement les oscillations et l'amplitude.

En un second temps, nous avons reconstitué avec python le système en question :



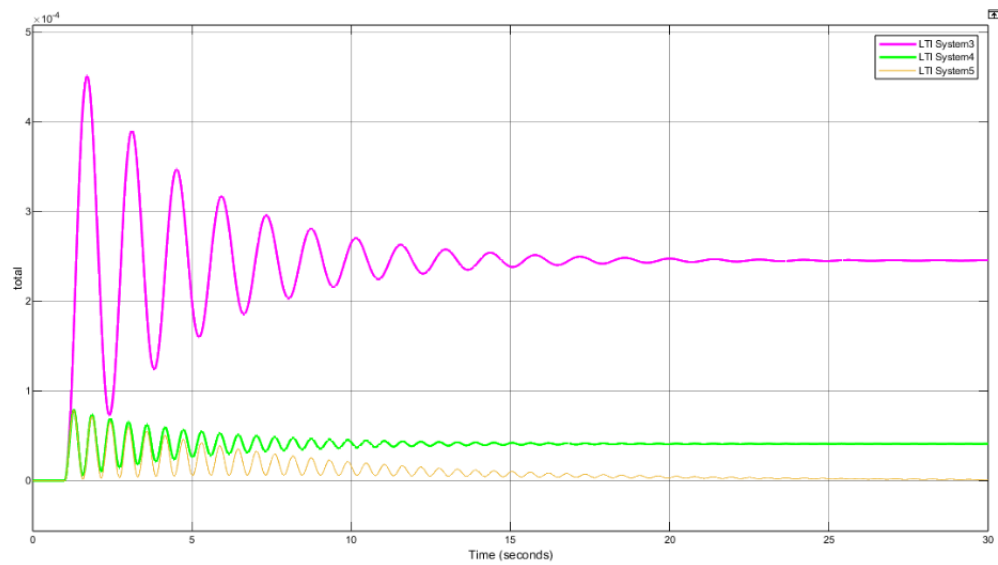
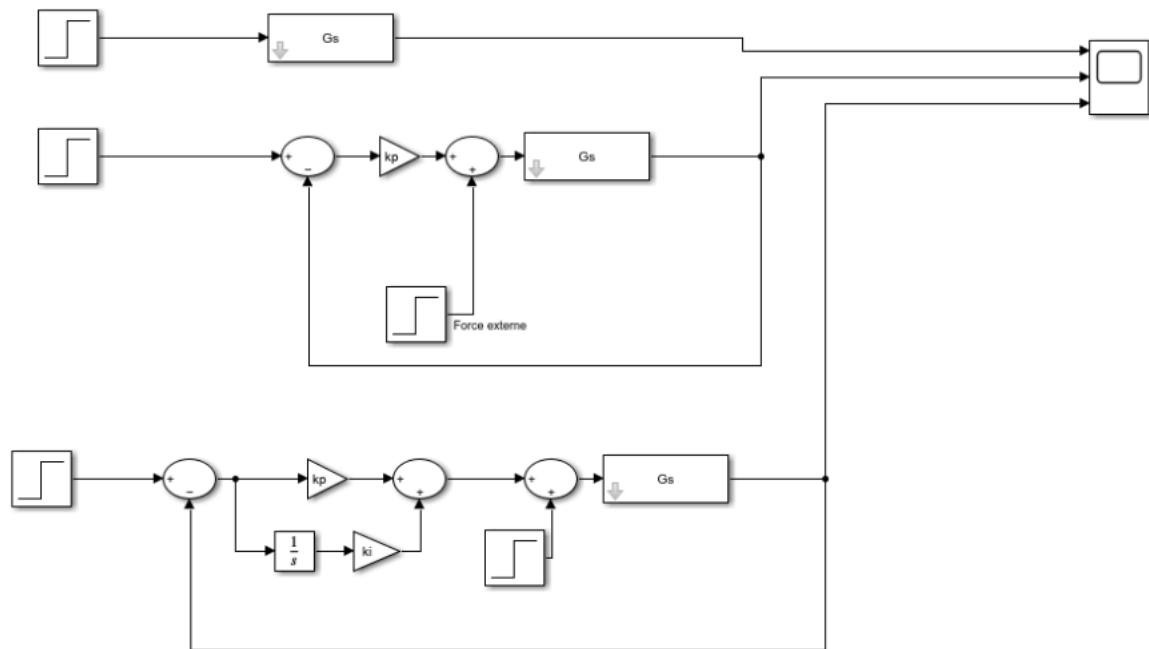
et constatons que lorsque k est faible donne une réponse lente. Les nombre d'oscillation avant stabilisation est faible et l'amplitude de ces dernieres sont grandes. Quand K augmente nous observons l'inverse. (nO++, A-)

4.4 D : Application d'un regulateur integrateur (PI)



Nous observons que le système se stabilise vers son état avant la perturbation. En d'autres termes, le regulateur integrateur annule l'erreur statique. Nous observons également une réponse plus lente, mais plus précise à long terme.

5 Conclusion



6 annexes

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
```

```
Paramètres du système
Kbobines = 1e-3 (N·m)/A
k = 2e-3 (N·m)/rad
b = 5e-5 (N·m)/(rad/s)
Jtot = 1e-4 kg·m2
l = 5e-2 m
g = 9.81 m/s2
m = 1e-3 kg (1 g)
```

```
Perturbation : ajout d'une masse à t = 1 s
def perturbation(t):
    return m * g * l if t >= 1 else 0
```

```
Équation différentielle du système en boucle fermée avec régulateur P
def systemode(theta, t, Kp):
    dthetadt = theta[1]
    Perturbation (couple dû à la masse)
    tauperturbation = perturbation(t)
    Couple des bobines : Kbobines * i, avec i = Kp * (-theta[0])
    taubobines = -Kbobines * Kp * theta[0]
    Équation différentielle : J * d2/dt2 = -k - b d/dt + bobines + perturbation
    d2thetadt2 = (-k * theta[0] - b * theta[1] + taubobines + tauperturbation) / Jtot
    return [dthetadt, d2thetadt2]
```

```
Temps de simulation
t = np.linspace(0, 30, 1000)
```

```
Simulation pour Kp = 1
solKp1 = odeint(systemode, [0, 0], t, args=(1,))
```

```
Simulation pour Kp = 100
solKp100 = odeint(systemode, [0, 0], t, args=(100,))
```

```
Tracé des résultats
plt.figure(figsize=(12, 6))
plt.plot(t, solKp1[:, 0], label='Kp = 1')
plt.plot(t, solKp100[:, 0], label='Kp = 100')
plt.axvline(x=1, color='gray', linestyle='--', label='Ajout de la masse (t=1s)')
plt.xlabel('Temps (s)')
```

```
plt.ylabel('Position angulaire (rad)')  
plt.title('Réponse du système pour  $K_p = 1$  et  $K_p = 100$ ')  
plt.legend()  
plt.grid(True)  
plt.show()
```